Performance Testing

Date	30 June 2026
Team ID	
Project Name	**A Comprehensive Measure of Well-Being**

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Web Browser
Type of Testing	Load Testing
Target Module / API	Flask Prediction Endpoint (/predict)
Test Environment	Local System / Render Deployment
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits prediction request	1	30	Prediction generated successfully
2	Multiple prediction requests	10	60	Application responds without errors
3	Continuous prediction requests	20	120	Stable response with low latency
4	Invalid input submission	5	30	Error handled gracefully without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.8 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 10 Req/sec	15 Req/sec	Pass	Handles multiple requests efficiently
4	Error Rate	< 1%	0%	Pass	No request failures observed
5	CPU Utilization	< 80%	42%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	48%	Pass	Memory usage remained within limits

Step 4: Observations & Analysis

- The Flask application responded quickly to prediction requests.
- The Linear Regression model generated predictions in under 2 seconds.
- No application crashes or server errors were observed during testing.
- CPU and memory usage remained well below the acceptable threshold.
- The deployed application maintained stable performance under normal load conditions.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):